Compressed and Sub-Line Cache Formats for BaM-Style Graph Storage

A Simulation Study of I/O Amplification, Device Operations, and Decode Cost

Shivamsh
Independent Researcher
shivamshsr@gmail.com

ABSTRACT

BaM gives GPUs on-demand access to NVMe storage through a software cache with fixed cache lines, and I/O amplification—bytes read per byte needed—is a central measure of its efficiency. We test two proposed improvements to raw 4 KB lines for graph neighbour lists: delta-plus-varint compressed lines, and adaptive lines that fetch 512 B sectors and upgrade after repeated sector misses. Using a deterministic software simulator, four graphs (including the real `web-Google` dataset), and 1,700 k-hop benchmark runs that all returned results identical to an in-memory reference, we find that both ideas reduce bytes but neither is a clear win. Compression reduces bytes and misses by only 1.3% without ID locality, 15–20% with it, and 31–51% with a cache holding half the graph; however, our NumPy decoder would need to be 2–48 times faster to break even on modelled total time. Adaptive lines cut bytes by 76–86% relative to raw 4 KB lines, yet a plain fixed 512 B line reads slightly less on all four graphs, and raw adaptive lines increase misses by 1.6–11.5%. Adaptive time benefit depends on the device: modelled I/O time falls 57–69% on Optane but rises 2–19% on NAND. The study measures bytes, operations, and hit rates only, not GPU throughput or contention. Its baseline is a reconstruction, and storage time is modelled rather than measured.

CCS CONCEPTS

• **Information systems** → **Information storage systems**.

KEYWORDS

GPU storage access, software caches, I/O amplification, graph compression, sectored caches

1 INTRODUCTION

Graph workloads routinely exceed GPU memory. BaM [9] addresses this by letting GPU threads issue NVMe requests directly, using a fine-grained software cache to coalesce accesses and to limit I/O amplification. Its graph evaluation uses 4 KB cache lines holding raw data. Fixed raw lines have an obvious cost: a short neighbour list that needs a few dozen bytes still pulls in an entire line.

Two remedies appear natural. Compressing neighbour lists packs more lists into each line, and compression is well established for graphs [1, 11]. Fetching sectors instead of whole lines avoids transferring unused bytes, an idea dating to sectored CPU caches [7]. This paper tests both in a controlled simulation:

H1 Delta-plus-varint compressed lines reduce bytes read and misses relative to raw 4 KB lines without affecting correctness.

H2 Adaptive lines, which fetch 512 B sectors and upgrade to a full line after N sector misses, reduce bytes read and misses relative to raw 4 KB lines without affecting correctness.

Because a format that saves bytes may still cost time, we also ask: **RQ1** when does compression pay for its decode cost? **RQ2** does adaptive fetching beat simply using a smaller fixed line? **RQ3** how sensitive are the conclusions to the storage device?

Contributions. (1) An open, deterministic harness comparing five stores behind one interface, validated against an in-memory reference. (2) A one-factor-at-a-time sweep over cache size, depth, line size, and upgrade threshold on synthetic and real graphs. (3) Evidence that H1 holds conditionally and H2 holds only for bytes, with break-even decode speeds that state what a practical decoder must achieve.

2 BACKGROUND

BaM.. BaM places a software cache in GPU memory and maps data to lines through a `bam::array` abstraction [9]. Threads in a warp coalesce accesses to the same line, and a single thread probes the cache for each unique line. Replacement uses a clock algorithm [2]: a global counter assigns slots and skips those pinned by a non-zero reference count.

I/O amplification. For a workload, we define

$$\text{Amp} = \frac{\text{bytes read from storage}}{\text{raw bytes requested by queries}}, \tag{1}$$

where the denominator counts 4 bytes per neighbour *for every format*, so a format cannot appear better merely by shrinking its own denominator. Each request counts, including repeats, so values below 1 are possible under heavy caching.

Device limits. BaM's own analysis shows that small accesses demand far higher request rates to reach a given bandwidth [9]. Bytes alone therefore do not determine cost: a design that reads fewer bytes through more requests can be slower on an IOPS-limited device.

3 DESIGN

All stores share one interface: `neighbors_batch(nodes)` returns neighbours in node order plus per-node counts, and `stats()` reports counters. Table 1 lists the configurations evaluated. Every cached store has the same cache budget in bytes, uses line-sized frames, and applies the same replacement and coalescing rules.

Table 1: Stores evaluated.

Name	Description
Load-all	Whole graph read once into RAM
Raw-4K	Baseline: raw `uint32` lists, fixed lines
Comp-S	Compressed, lists may straddle lines
Comp-P	Compressed, padded to avoid straddling
Adapt-R	Adaptive fetch over raw lists
Adapt-C	Adaptive fetch over compressed lists

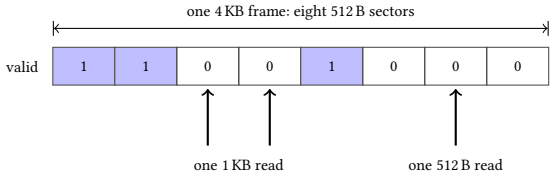


Figure 1: Adaptive frame. Needed sectors 2, 3, and 6 cost two device operations, since each contiguous run is one read.

3.1 Compressed line format

Each neighbour list is sorted. Its first identifier is stored as a zigzag-encoded offset from the source node, and the remainder as gaps, all as little-endian base-128 varints. Per-node byte offsets remain in RAM. Degrees need not be stored: a list's value count equals the number of terminator bytes (high bit clear) in its byte range. With straddling, a list's size follows from consecutive offsets, so index RAM matches the baseline ($8(n+1)$ bytes). Padding moves a list that would cross a boundary to the next line, which requires 4 more bytes per node to recover list lengths. Lists longer than one line necessarily span lines.

Decoding is vectorised with NumPy. Terminators delimit values; payload bytes are shifted by position and summed with `add.reduceat`; a global cumulative sum, corrected at list heads, reconstructs identifiers. Lines are cached compressed and decoded on read.

3.2 Adaptive lines

Adaptive stores keep line-sized frames with a per-frame sector-valid bitmap (Figure 1). On a probe, the warp's required sectors are compared with the bitmap and only absent ones are read, one device operation per contiguous run. Each frame counts missed sectors since it was installed. When the count reaches threshold N, all remaining sectors are fetched. Therefore $N \leq 1$ is a full-line fetch, and a very large N disables upgrades. A partly filled frame still occupies a full line, so sparsely used frames waste capacity. A probe needing any absent sector counts as a miss.

4 METHODOLOGY

4.1 Harness and baseline

The harness is Python with NumPy; data files are real and are read with `pread`. **The baseline is a reconstruction.** The original toy simulator was not available, so we re-implemented on-disk CSR, a second-chance clock cache, and warp-style coalescing from the BaM paper [9] and a written specification. Second-chance replacement differs from BaM's counter-only clock. Warps are simulated as groups of 32 list requests.

Table 2: Graphs. Raw size is 4 bytes per edge. "Size" is compressed relative to raw; "Pad" is added file size from padding.

			Bytes/edge			
Graph	Nodes	Edges	Strad.	Pad.	Size	Pad
`random`	1,000,000	15,999,870	2.77	2.78	69%	0.6%
`powerlaw`	1,000,000	11,347,768	1.56	1.63	39%	4.4%
`web-Google`	875,713	5,105,039	2.80	2.81	70%	0.4%
`web-Google-BFS`	875,713	5,105,039	1.86	1.87	47%	0.2%

4.2 Graphs

Table 2 summarises the inputs. `random` is a uniform directed graph. `powerlaw` is synthetic: truncated-Pareto out-degrees, with 80% of edges landing near the source at heavy-tailed ID distance and the rest on popular global targets. Parameters are ours and are not fitted to a real graph. `web-Google` is a real web graph from SNAP [6] using its numeric ID order; `web-Google-BFS` is the same graph relabelled by breadth-first traversal, in the spirit of bandwidth-reducing orderings [3, 12]. All graphs are sorted and deduplicated.

4.3 Workload and parameters

Each run starts with a cold cache and executes 2,000 k-hop queries from uniformly random seeds, one batch per BFS level. Five query seeds are used; synthetic graphs are fixed at generator seed 0. The default configuration is a cache of 2% of raw graph size, depth 2, 4 KB lines, $N = 3$, and sorted frontiers. We vary one factor at a time: cache size (0.1–50%), depth (1–3), line size (512 B–64 KB), N (0–99), and frontier order (sorted or shuffled).

4.4 Metrics and device model

We count storage bytes, misses, hits, coalesced accesses, and the size of every device read. Hit rate is hits per unique line probe. Confidence intervals are two-sided 95% Student-t intervals over paired per-seed differences ($n = 5$). Because reads come from the OS page cache, measured wall time contains no device cost and is dominated by Python simulation overhead. We therefore *model* storage time, summing a per-operation cost:

$$t_{\mathrm{NAND}}(s) = \max\left(\frac{1}{750\,\mathrm{K}},\ \frac{s}{7\,\mathrm{GB/s}}\right), \tag{2}$$

$$t_{\mathrm{Optane}}(s) = \frac{1}{5.1\,\mathrm{M}} + \max(s-512,0)\,\frac{1/1.5\,\mathrm{M} - 1/5.1\,\mathrm{M}}{3584}. \tag{3}$$

Rates come from BaM's device table [9] (980 Pro: 700–800 K read IOPS; P5800X: 5.1 M at 512 B, 1.5 M at 4 KB). The 7 GB/s NAND transfer limit is our assumption for PCIe Gen4 x4. This is a throughput model of one device, without queueing or latency.

Decode time is measured in two ways. *Measured* decode is the in-run time, with one call per frontier batch. *Bulk* decode applies a sequentially calibrated rate to the edge count, excluding per-call

overhead. The calibrated decoder runs at 38–52 ns/edge, with 19–23 μs for a 16-list batch. We report bulk decode where possible, since it favours compression.

4.5 Correctness and reproducibility

A pytest suite of 423 cases compares every store with both load-all and an independent pure-Python BFS across multiple seeds, graphs, depths, cache sizes, and line sizes. Edge cases include degree-0 nodes, reads past end of file, and a hub whose list exceeds the entire cache. An independent scalar decoder cross-checks the NumPy decoder. A differential test requires the new cache code in raw, full-line mode to reproduce the baseline's counters exactly, so format comparisons are not artefacts of reimplementation. At runtime, a hash of every query result was compared with load-all for all 1,700 runs, with no mismatch. Two complete benchmark executions produced identical bytes, misses, hits, and operation counts in all 340 summary rows. Timings are not deterministic: mean decode time for a configuration differed by up to 24% between the runs.

5 RESULTS

Table 3 gives absolute results at the default point, and Table 4 gives paired changes against Raw-4K.

5.1 H1: compression helps conditionally

Without ID locality, compression barely helps. `random` and `web-Google` shrink to about 70% of raw size, yet Comp-S reads only 1.3% fewer bytes: under sparse cold access, each list still costs about one line, whatever the line's density. Benefits appear when a line's additional lists are actually used. With locality, bytes and misses fall 15–20%, and coalescing on `web-Google-BFS` rises from 28.5% to 34.1%. Benefits also grow with cache size (Figure 2), because a fixed byte budget holds more of the graph. At a 50% cache, misses fall 31% on `random` and `web-Google`, 50% on `powerlaw`, and 51% on `web-Google-BFS`. H1 is supported for bytes and misses, with strongly conditional effect size.

Reordering matters as much as encoding. BFS relabelling alone reduces Raw-4K bytes by 36% at the default point, more than compression achieves on the same graph in that configuration. Padding has little effect: at most about one percentage point in bytes, at the cost of 0.2–4.4% more file and 50% more index RAM.

5.2 RQ1: decode cost dominates compression's I/O savings

Table 5 gives the decode speed at which Comp-S breaks even with Raw-4K. Our decoder is slower than break-even everywhere shown: the required speedup on NAND ranges from about 2× with a large cache to 48× on `random` at the default point. Figure 3 shows the resulting modelled total time. With bulk decode, plain compression beat the baseline only at 64 KB lines (at most 6 of 280 paired runs per store), an impractical size at which raw reads are themselves inflated. With measured decode, it never won. Encoding cost is small, at 0.3–1.1 s per graph.

5.3 H2 and RQ2: adaptive lines against smaller lines

Adapt-R cuts bytes by 76–86%, so H2 holds for bytes against 4 KB lines. It fails for misses: Adapt-R incurs 1.6–11.5% more, and hit rate drops sharply (0.126 to 0.050 on `web-Google-BFS`). Combining formats helps only modestly: Adapt-C has about 5% fewer misses on graphs with locality, still with more device operations, and reads 3–13% fewer bytes than Adapt-R.

The stronger comparison is with a smaller fixed line (Table 6, Figure 4). Raw 512 B lines give lower amplification than Adapt-R on all four graphs, 8.5–11.1 against 9.0–11.7. Adapt-R's remaining advantages are 3–20% fewer device operations than 512 B lines, and one eighth as many cache tags, which we did not include in the RAM accounting.

The upgrade rule did not earn its cost in bytes. Amplification falls monotonically as N grows and is lowest when upgrades never fire (Figure 5). Upgrades also explain a counter-intuitive result: on `random`, Adapt-R amplification rises from 9.0 to 12.2 as the cache grows from 0.1% to 50%, because lines stay resident long enough to accumulate misses, taking upgrades from 4 to 3,768.

5.4 RQ3: device sensitivity

The time verdict on Adapt-R reverses with the device model. On Optane, modelled I/O falls 57–69% since small reads are genuinely cheaper. On NAND, it rises 2–19%, because a 512 B read occupies the same IOPS slot as a 4 KB read and adaptive issues more operations. Across all sweeps, Adapt-R beat the baseline on total time in 260 of 280 Optane runs but in only 40 of 280 NAND runs, all of the latter at 16 KB lines or larger.

5.5 On-demand versus load-all

Table 7 shows that deeper traversals favour load-all on this workload. At depth 3, load-all reads the fewest bytes on all four graphs. This reflects how much of the graph 2,000 queries reach. Load-all requires the entire graph in memory, the constraint BaM exists to remove, so this is a statement about bytes only, not a recommendation.

6 DISCUSSION

Three practical implications follow. First, ordering should come before encoding: relabelling is free at query time and it benefits every format. Second, sub-line fetching is most of the way achievable with a smaller fixed line, so a more complex adaptive scheme needs to justify itself through operation counts or tag overhead rather than bytes. Third, compression is worth revisiting only with a much faster decoder and a cache that holds a meaningful fraction of the graph. SIMD integer decoders achieve very high throughput [5], and such a decoder might reach the break-even range in Table 5, but we did not measure one, and a GPU decoder would face its own constraints.

7 THREATS TO VALIDITY

Internal validity. The baseline is a reconstruction and has not been compared with the original simulator. Its replacement policy differs from BaM's. Differential tests show that new and baseline

Table 3: Default point: cache 2% of raw graph, depth 2, 4 KB lines, $N = 3$; means over five query seeds. Amp shown ± one standard deviation. NAND and Optane are modelled. Raw-4K decode is reported as 0 because its copy runs inside the baseline, untimed; Adapt-R's value is that same copy.

Store	Storage MB	Amp	Misses	Hit rate	Coalesced %	Decode ms (measured)	Decode ms (bulk)	Wall s	NAND I/O ms	Optane I/O ms
`random`										
Load-all	64.0	29.5±0.1	–	–	–	0	0.0	0.32	9.1	8.4
Raw-4K	138.3	63.7±0.1	33,772	0.020	0.0	0	0.0	0.45	45.0	22.5
Comp-S	136.5	62.9±0.1	33,334	0.028	0.1	135	28.0	0.59	44.4	22.2
Comp-P	135.1	62.2±0.1	32,984	0.028	0.1	135	28.0	0.59	44.0	22.0
Adapt-R	19.6	9.0±0.0	34,360	0.003	0.0	6	0.0	0.48	45.9	7.0
Adapt-C	19.1	8.8±0.0	34,170	0.004	0.1	136	28.0	0.61	45.7	6.9
`powerlaw`										
Load-all	45.4	41.4±5.1	–	–	–	0	0.0	0.33	6.5	6.0
Raw-4K	52.6	47.6±3.1	12,854	0.123	42.0	0	0.0	0.38	17.1	8.6
Comp-S	42.3	38.2±2.3	10,326	0.193	49.0	132	10.5	0.52	13.8	6.9
Comp-P	42.4	38.3±2.2	10,354	0.191	48.9	132	10.5	0.52	13.8	6.9
Adapt-R	12.9	11.7±0.8	14,319	0.022	42.0	6	0.0	0.42	20.4	3.7
Adapt-C	11.3	10.2±0.8	12,300	0.038	49.0	136	10.5	0.55	18.0	3.2
`web-Google`										
Load-all	20.4	34.1±0.8	–	–	–	0	0.0	0.27	2.9	2.7
Raw-4K	50.6	84.6±0.7	12,352	0.022	0.1	0	0.0	0.35	16.5	8.2
Comp-S	49.9	83.5±0.6	12,189	0.030	0.2	119	7.4	0.47	16.3	8.1
Comp-P	49.5	82.8±0.6	12,089	0.031	0.1	118	7.4	0.47	16.1	8.1
Adapt-R	7.0	11.7±0.1	12,555	0.006	0.1	6	0.0	0.36	16.8	2.5
Adapt-C	6.8	11.4±0.1	12,474	0.007	0.2	118	7.4	0.48	16.7	2.5
`web-Google-BFS`										
Load-all	20.4	34.0±0.9	–	–	–	0	0.0	0.27	2.9	2.7
Raw-4K	32.5	54.0±0.9	7,934	0.126	28.5	0	0.0	0.31	10.6	5.3
Comp-S	27.7	46.1±0.8	6,767	0.185	34.1	111	6.3	0.41	9.0	4.5
Comp-P	27.6	45.9±0.9	6,738	0.185	34.1	110	6.3	0.40	9.0	4.5
Adapt-R	6.5	10.8±0.2	8,625	0.050	28.5	6	0.0	0.30	12.2	2.0
Adapt-C	6.0	9.9±0.2	7,547	0.092	34.1	111	6.3	0.42	11.0	1.8

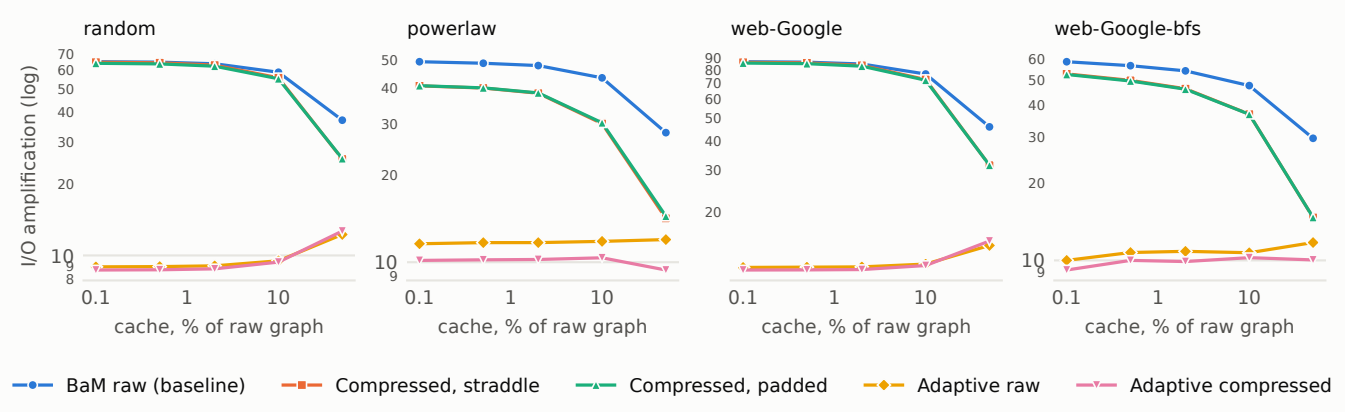


Figure 2: I/O amplification against cache size (depth 2, 4 KB lines; log scales). Comp-S and Comp-P nearly coincide. Adaptive lines sit far below at small caches, while compression closes the gap as the cache grows.

cache code agree with each other, not that either matches BaM. Decode times were taken with up to eight concurrent workers on 24 cores, and differed by up to 24% between runs; we do not rely on small timing differences.

Construct validity. Storage time comes from a simple model, and RQ3's outcome depends on it. Real arrays, queueing, and PCIe limits could change the balance; link-bandwidth bottlenecks

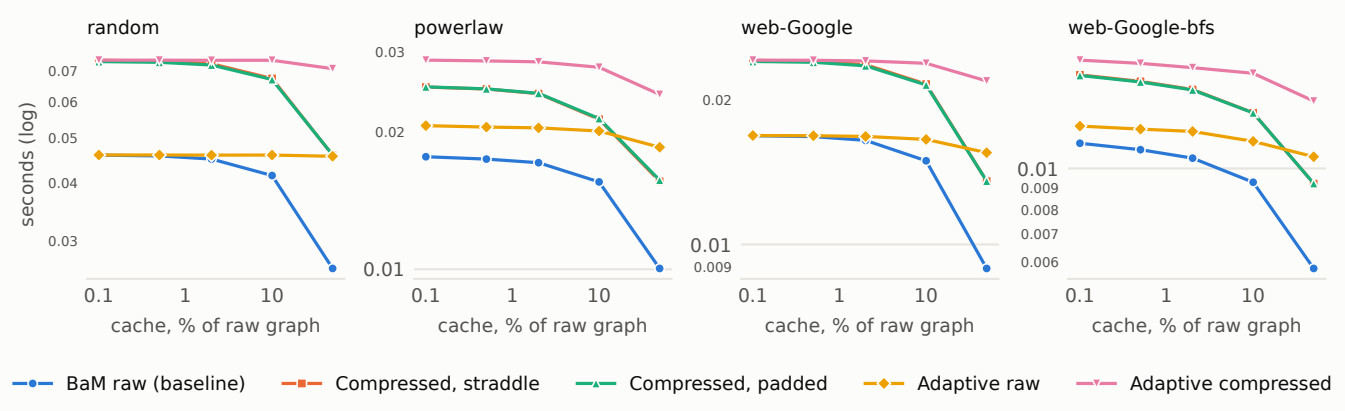


Figure 3: Modelled NAND I/O time plus bulk decode, in seconds (log scales). The raw baseline is fastest at every cache size; Adapt-R matches it only at small caches on graphs without locality.

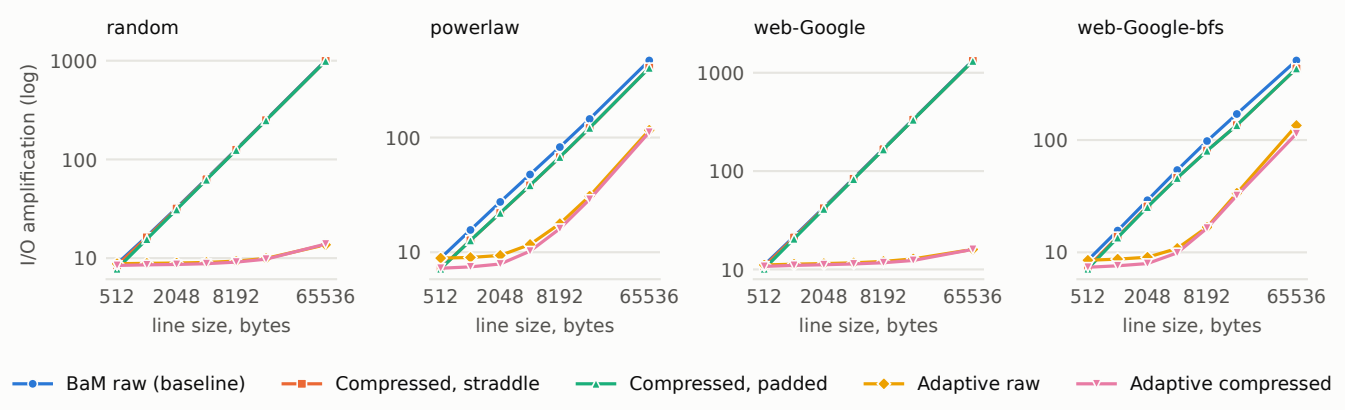


Figure 4: I/O amplification against line size (cache 2%, depth 2; log scales). Sector granularity stays fixed at 512 B for adaptive stores, so raw and adaptive curves converge at 512 B.

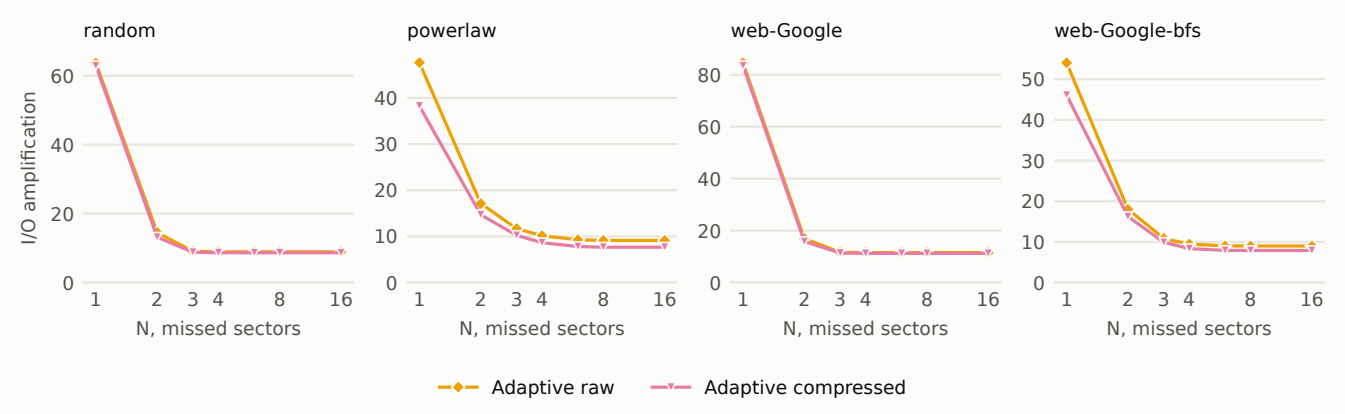


Figure 5: Adaptive amplification against upgrade threshold N. $N = 1$ denotes a full-line fetch and 16 denotes upgrades disabled (measured with $N = 99$).

would favour adaptive fetching. Hit rates are not directly comparable across formats, since stores probe different lines and adaptive partial hits count as misses. Tag and bitmap memory

Table 4: Change (%) relative to Raw-4K at the default point, with paired 95% CI ($n = 5$). Negative is better.

Store	Bytes	Misses	Dev. ops	NAND I/O	Optane I/O
`random`					
Comp-S	−1.3 ± 0.1	−1.3 ± 0.1	−1.3 ± 0.1	−1.3 ± 0.1	−1.3 ± 0.1
Comp-P	−2.3 ± 0.2	−2.3 ± 0.2	−2.3 ± 0.2	−2.3 ± 0.2	−2.3 ± 0.2
Adapt-R	−85.8 ± 0.1	+1.7 ± 0.0	+2.0 ± 0.1	+2.0 ± 0.1	−68.8 ± 0.1
Adapt-C	−86.2 ± 0.0	+1.2 ± 0.1	+1.5 ± 0.2	+1.5 ± 0.2	−69.3 ± 0.1
`powerlaw`					
Comp-S	−19.7 ± 0.8	−19.7 ± 0.8	−19.7 ± 0.8	−19.7 ± 0.8	−19.7 ± 0.8
Comp-P	−19.5 ± 0.9	−19.5 ± 0.9	−19.5 ± 0.9	−19.5 ± 0.9	−19.5 ± 0.9
Adapt-R	−75.5 ± 0.7	+11.5 ± 0.9	+19.3 ± 1.0	+19.3 ± 1.0	−57.2 ± 0.7
Adapt-C	−78.6 ± 0.5	−4.3 ± 0.9	+5.1 ± 1.0	+5.1 ± 1.0	−62.4 ± 0.6
`web-Google`					
Comp-S	−1.3 ± 0.2	−1.3 ± 0.2	−1.3 ± 0.2	−1.3 ± 0.2	−1.3 ± 0.2
Comp-P	−2.1 ± 0.3	−2.1 ± 0.3	−2.1 ± 0.3	−2.1 ± 0.3	−2.1 ± 0.3
Adapt-R	−86.2 ± 0.1	+1.6 ± 0.1	+1.9 ± 0.2	+1.9 ± 0.2	−69.2 ± 0.1
Adapt-C	−86.6 ± 0.1	+1.0 ± 0.2	+1.3 ± 0.2	+1.3 ± 0.2	−69.6 ± 0.1
`web-Google-BFS`					
Comp-S	−14.7 ± 0.5	−14.7 ± 0.5	−14.7 ± 0.5	−14.7 ± 0.5	−14.7 ± 0.5
Comp-P	−15.1 ± 0.7	−15.1 ± 0.7	−15.1 ± 0.7	−15.1 ± 0.7	−15.1 ± 0.7
Adapt-R	−79.9 ± 0.2	+8.7 ± 0.5	+15.7 ± 0.9	+15.7 ± 0.9	−61.5 ± 0.2
Adapt-C	−81.7 ± 0.2	−4.9 ± 0.8	+3.6 ± 0.9	+3.6 ± 0.9	−65.2 ± 0.3

Table 5: Break-even decode speed for Comp-S (ns/edge; higher is easier), and the speedup needed on NAND relative to our calibrated decoder.

		Cache 2%			Cache 50%		
Graph	Ours	NAND	Opt.	Need	NAND	Opt.	Need
`random`	51.5	1.1	0.5	47.9×	15.1	7.5	3.4×
`powerlaw`	37.9	12.1	6.1	3.1×	17.9	9.0	2.1×
`web-Google`	49.6	1.4	0.7	34.2×	18.6	9.3	2.7×
`web-Google-BFS`	42.2	10.3	5.2	4.1×	19.5	9.8	2.2×

are not accounted for. Nothing here measures GPU throughput, contention, or latency, and a 4 KB-formatted drive would preclude sub-4 KB reads.

External validity. We used one real graph of about 5 million edges, smaller than the graphs BaM targets, and one instance of each synthetic graph. Our power-law locality parameters are unvalidated. The workload is uniform-seed k-hop traversal without concurrency; skewed query popularity could increase cache value and could help compression. Sweeps vary one factor at a time, so interactions are largely untested, and the run counts reported above depend on which points were swept.

8 RELATED WORK

BaM [9] supplies the architecture studied here, and Active-Pointers [10] is earlier work on demand paging from GPUs. WebGraph [1] established compression that exploits locality in web graphs, and Ligra+ [11] showed that compressed graphs can be processed in parallel in memory; our work instead asks how compression interacts with a storage-backed line cache. Fast integer decoding is studied by Lemire and Boytsov [5]. Cache compression has been explored for on-chip caches, for example Base-Delta-Immediate [8]. Sectored fetch dates to the System/360 Model 85 [7], and Footprint Cache [4] applies selective block fetching to die-stacked DRAM caches. Graph reordering is treated by Cuthill and McKee [3] and by Gorder [12].

Table 6: Adaptive 4 KB lines against fixed raw lines at the default cache size. I/O in ms, modelled.

Store	Amp	Hit rate	Misses	Dev. ops	NAND	Optane
`random`						
Raw-4K	63.71	0.020	33,772	33,772	45.03	22.51
Raw-1K	16.64	0.019	35,283	35,283	47.04	9.29
Raw-512	8.78	0.020	37,222	37,222	49.63	7.30
Adapt-R	9.04	0.003	34,360	34,442	45.92	7.02
`powerlaw`						
Raw-4K	47.64	0.123	12,854	12,854	17.14	8.57
Raw-1K	15.64	0.063	16,869	16,869	22.49	4.44
Raw-512	8.89	0.051	19,170	19,170	25.56	3.76
Adapt-R	11.68	0.022	14,319	15,330	20.44	3.67
`web-Google`						
Raw-4K	84.57	0.022	12,352	12,352	16.47	8.23
Raw-1K	21.59	0.033	12,615	12,615	16.82	3.32
Raw-512	11.09	0.043	12,963	12,963	17.28	2.54
Adapt-R	11.67	0.006	12,555	12,585	16.78	2.54
`web-Google-BFS`						
Raw-4K	54.03	0.126	7,934	7,934	10.58	5.29
Raw-1K	15.63	0.111	9,178	9,178	12.24	2.42
Raw-512	8.50	0.108	9,985	9,985	13.31	1.96
Adapt-R	10.84	0.050	8,625	9,175	12.23	2.04

Table 7: Storage MB read by depth, with the smallest in bold. "Need" is raw MB requested.

Graph	k	Need	Load-all	Raw-4K	Comp-S	Adapt-R	Adapt-C
`random`	1	0.13	64.0	8.1	8.1	1.2	**1.1**
	2	2.17	64.0	138.3	136.5	19.6	**19.1**
	3	34.92	**64.0**	2221.4	2191.2	315.7	306.7
`powerlaw`	1	0.09	45.4	8.1	7.8	1.1	**1.1**
	2	1.11	45.4	52.6	42.3	12.9	**11.3**
	3	13.01	**45.4**	435.3	340.0	121.4	95.9
`web-Google`	1	0.05	20.4	6.8	6.8	0.9	**0.9**
	2	0.60	20.4	50.6	49.9	7.0	**6.8**
	3	3.69	**20.4**	278.9	274.9	39.0	37.8
`web-Google-BFS`	1	0.05	20.4	6.8	6.6	0.9	**0.9**
	2	0.60	20.4	32.5	27.7	6.5	**6.0**
	3	3.60	**20.4**	111.3	77.7	39.7	33.1

9 CONCLUSION

Neither format is a straightforward replacement for raw 4 KB lines. Compression reliably reduces bytes and misses, by an amount governed by locality and cache size, but our decoder gives back more time than the I/O saves. Adaptive lines greatly reduce bytes but raise misses in raw form, are matched by fixed 512 B lines on bytes,

and help time only on devices where small reads are cheap. Future work should compare against the original baseline, evaluate a native decoder, test real devices and skewed workloads, include larger graphs such as soc-LiveJournal1, and account for metadata costs.

DATA AND ARTIFACT AVAILABILITY

The code, per-seed results (`results/runs.csv`), and table generators accompany this paper in the project repository. `web-Google` is available from the SNAP collection. The benchmark ran on Linux with Python 3.14 and NumPy 2.5.4.

ACKNOWLEDGMENTS

Generative AI (Claude Opus 5.5, Anthropic) was used to implement the harness, run the experiments, analyse results, and draft this manuscript. The author is responsible for reviewing and validating the work. No external funding was received.

REFERENCES

[1] Paolo Boldi and Sebastiano Vigna. 2004. The WebGraph Framework I: Compression Techniques. In *Proceedings of the 13th International Conference on World Wide Web (WWW '04)*. ACM, 595–602. https://doi.org/10.1145/988672.988752

[2] F. J. Corbató. 1968. *A Paging Experiment with the Multics System.* Technical Report. Massachusetts Institute of Technology, Project MAC.

[3] E. Cuthill and J. McKee. 1969. Reducing the Bandwidth of Sparse Symmetric Matrices. In *Proceedings of the 1969 24th National Conference*. ACM, 157–172. https://doi.org/10.1145/800195.805928

[4] Djordje Jevdjic, Stavros Volos, and Babak Falsafi. 2013. Die-Stacked DRAM Caches for Servers: Hit Ratio, Latency, or Bandwidth? Have It All with Footprint Cache. In *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA '13)*. ACM, 404–415. https://doi.org/10.1145/2485922.2485957

[5] Daniel Lemire and Leonid Boytsov. 2015. Decoding Billions of Integers per Second through Vectorization. *Software: Practice and Experience* 45, 1 (2015), 1–29. https://doi.org/10.1002/spe.2203

[6] Jure Leskovec, Kevin J. Lang, Anirban Dasgupta, and Michael W. Mahoney. 2009. Community Structure in Large Networks: Natural Cluster Sizes and the Absence of Large Well-Defined Clusters. *Internet Mathematics* 6, 1 (2009), 29–123. https://doi.org/10.1080/15427951.2009.10129177

[7] J. S. Liptay. 1968. Structural Aspects of the System/360 Model 85, II: The Cache. *IBM Systems Journal* 7, 1 (1968), 15–21. https://doi.org/10.1147/sj.71.0015

[8] Gennady Pekhimenko, Vivek Seshadri, Onur Mutlu, Phillip B. Gibbons, Michael A. Kozuch, and Todd C. Mowry. 2012. Base-Delta-Immediate Compression: Practical Data Compression for On-Chip Caches. In *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques (PACT '12)*. ACM, 377–388. https://doi.org/10.1145/2370816.2370870

[9] Zaid Qureshi, Vikram Sharma Mailthody, Isaac Gelado, Seungwon Min, Amna Masood, Jeongmin Park, Jinjun Xiong, C. J. Newburn, Dmitri Vainbrand, I-Hsin Chung, Michael Garland, William Dally, and Wen-mei Hwu. 2023. GPU-Initiated On-Demand High-Throughput Storage Access in the BaM System Architecture. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '23)*. ACM, 325–339. https://doi.org/10.1145/3575693.3575748

[10] Sagi Shahar, Shai Bergman, and Mark Silberstein. 2016. ActivePointers: A Case for Software Address Translation on GPUs. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA '16)*. IEEE, 596–608. https://doi.org/10.1109/ISCA.2016.58

[11] Julian Shun, Laxman Dhulipala, and Guy E. Blelloch. 2015. Smaller and Faster: Parallel Processing of Compressed Graphs with Ligra+. In *2015 Data Compression Conference*. IEEE, 403–412. https://doi.org/10.1109/DCC.2015.8

[12] Hao Wei, Jeffrey Xu Yu, Can Lu, and Xuemin Lin. 2016. Speedup Graph Processing by Graph Ordering. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD '16)*. ACM, 1813–1828. https://doi.org/10.1145/2882903.2915220